International Institute of Information Technology, Hyderabad

Distributed Systems • Monsoon 2026 • Project Write-up

Dynamo-Style Distributed Key-Value Store

A leaderless, fault-tolerant, eventually consistent storage system

Members: Prateek Tiwari (2025201037) | Meet Ghelani (2025201096)

1. Project Description

Modern large-scale services such as shopping carts and session stores must remain *always writable*, even when servers fail or the network partitions. Amazon's Dynamo [1] showed that this can be achieved by giving up strict consistency in favour of high availability. This project designs and implements a Dynamo-style distributed key-value store that exposes a simple `put`/`get`/`delete` interface over gRPC and runs as a five-node cluster.

The system is fully **leaderless**: every node is identical and any node can coordinate a request for any key. Keys are partitioned with **consistent hashing** using virtual nodes and replicated on N=3 distinct nodes. Clients tune the consistency–latency trade-off through read and write quorums (R, W). **Vector clocks** detect concurrent updates, a **gossip protocol** provides decentralised membership and failure detection, and **sloppy quorums with hinted handoff** keep the cluster writable during outages. Each node persists data in a write-ahead log so acknowledged writes survive crashes. Figure 1 shows the request path.

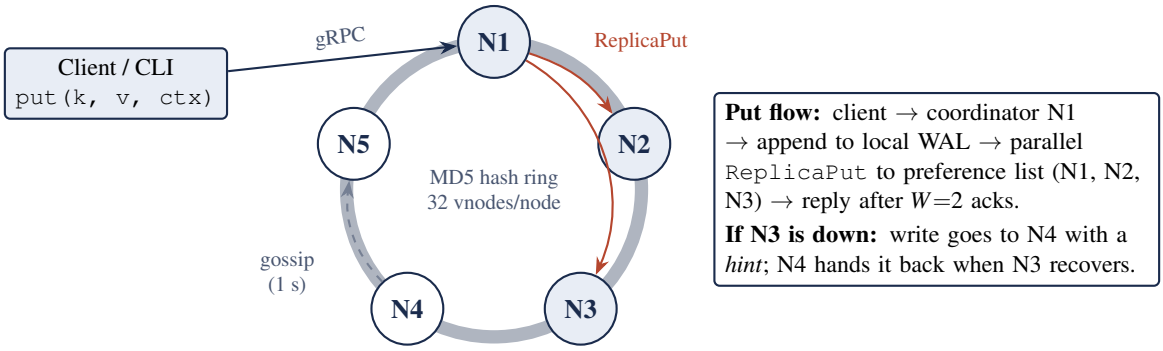


Figure 1: System overview: a 5-node leaderless ring with N=3 replicas (shaded) and quorum W=2.

2. Goals

- **Scalable partitioning:** distribute keys evenly using consistent hashing with virtual nodes, so that adding or removing a node remaps only about $1/n$ of the keys.
- **Replication with tunable consistency:** store every key on N=3 nodes and support per-request quorums (default $N/R/W = 3/2/2$, so that $R+W > N$ guarantees overlapping read and write sets).
- **Correct conflict handling:** track causality with vector clocks and keep concurrent versions as *siblings* for client-side reconciliation instead of silently losing writes.
- **High availability and self-healing:** detect failures with push–pull gossip (ALIVE → SUSPECT → DEAD), stay writable via sloppy quorum and hinted handoff, and converge stale replicas using read repair.
- **Durability:** guarantee that an acknowledged write survives a crash using an `fsync`'d write-ahead log, periodic snapshots and tombstones for deletes.
- **Empirical evaluation:** measure throughput and tail latency across quorum settings and cluster sizes, and validate correctness under injected failures.

3. Tools and Technologies

Category	Tools / Technologies	Role in the project
Language	Python 3.11	Implementation of nodes, client and test tools
Concurrency	`asyncio`	Non-blocking request handling; parallel fan-out of replica RPCs
Communication	gRPC (`grpcio`), Protocol Buffers	Client API and node-to-node RPCs (replication, forwarding, gossip, hints)
Storage	Write-ahead log (JSON lines), snapshots	Per-node persistence and crash recovery
Deployment	Docker, Docker Compose, YAML	Reproducible 5-node cluster and failure injection
Testing	`pytest`, custom chaos script	Unit tests, cluster scenarios, random node kill/restart
Evaluation	Custom load generator, `matplotlib`	Throughput and p50/p99 latency measurement and plots
Collaboration	Git, GitHub	Version control and code review

4. Special System Requirements

No additional computing resources are needed.

5. Expected Outcomes

Deliverables.

- A working five-node Dynamo-style key-value store with a gRPC API and a command-line client (`kv put`, `get`, `delete`, `info`) that transparently carries version context.
- Full implementation of the core mechanisms: consistent hashing, N-way replication, tunable quorums, vector clocks, read repair, gossip-based failure detection, sloppy quorum with hinted handoff, and WAL-based durability.
- An automated test suite (unit tests + Docker-based cluster scenarios for crashes, restarts, partitions and concurrent writes), a benchmark report with plots, and a documented GitHub repository.
- A live demonstration in which reads and writes continue while nodes are killed and recovered.

Expected results.

- Zero acknowledged writes lost when nodes are repeatedly crashed and restarted under live load.
- Continued read/write availability with one node down ($N{=}3$, $R{=}W{=}2$); failed nodes detected automatically within $\sim$10 s, with missed writes delivered on rejoin.
- Benchmarks that quantify the latency–consistency trade-off between fast ($R{=}W{=}1$) and overlapping ($R{=}W{=}2$) quorums, and throughput as the cluster grows from 1 to 3 to 5 nodes.

Reference. [1] G. DeCandia *et al.*, "Dynamo: Amazon's Highly Available Key-Value Store," in *Proc. 21st ACM Symposium on Operating Systems Principles (SOSP)*, 2007, pp. 205–220.